UK Atomic Energy Authority

A tour of OpenFOAM

Matthew Falcone

Overview

UK Atomic Energy Authority

- Introduction
  - Background and .org vs .com
  - .org's module system
  - Case directory structure

- Flow over a heated plate: a quick tutorial
  - blockMesh, decomposePar
  - foamRun

OpenFOAM

UK Atomic Energy Authority

- OpenFOAM is an open-source CFD toolkit based on the finite volume method
  - Solvers constructed from classes and functions within the toolkit
  - OpenFOAM packaged with many pre-existing solvers
- Two flavours of OpenFOAM
  - OpenFOAM Foundation (.org)
    - What Hippo is based on
    - Uses a solver module system
  - ESI-OpenCFD (.com)
    - Probably more popular for developers
    - Uses many standalone solvers

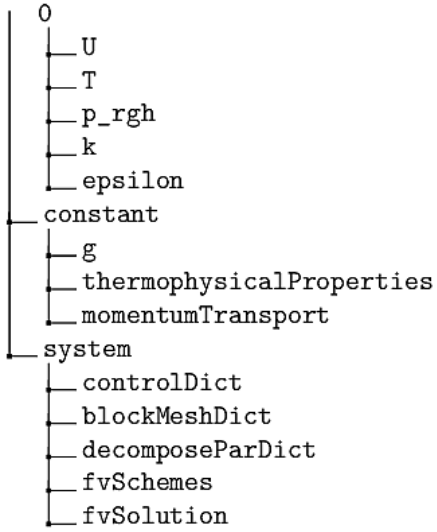
```
void Foam::solvers::isothermalFluid::momentumPredictor()
{
    volVectorField& U(U_);

    tUEqn =
    (
        fvm::ddt(rho, U) + fvm::div(phi, U)
      + MRF.DDt(rho, U)
      + momentumTransport->divDevTau(U)
     ==
        fvModels().source(rho, U)
    );
    fvVectorMatrix& UEqn = tUEqn.ref();

    UEqn.relax();

    fvConstraints().constrain(UEqn);

    if (pimple.momentumPredictor())
    {
        if (buoyancy.valid())
        {
            solve
            (
                UEqn
             ==
                netForce()
            );
        }
        else
        {
            solve(UEqn == -fvc::grad(p));
        }

        fvConstraints().constrain(U);
        K = 0.5*magSqr(U);
    }
}
```

```
int main(int argc, char *argv[])
{
    #include "setRootCase.H"
    #include "createTime.H"
    #include "createMesh.H"

    pisoControl piso(mesh);

    #include "createFields.H"
    #include "initContinuityErrs.H"

    // * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //

    Info<< "\nStarting time loop\n" << endl;

    while (runTime.loop())
    {
        Info<< "Time = " << runTime.userTimeName() << nl << endl;

        #include "CourantNo.H"

        // Momentum predictor

        fvVectorMatrix UEqn
        (
            fvm::ddt(U)
          + fvm::div(phi, U)
          - fvm::laplacian(nu, U)
        );

        if (piso.momentumPredictor())
        {
            solve(UEqn == -fvc::grad(p));
        }

        // --- PISO loop
        while (piso.correct())
        {
            volScalarField rAU(1.0/UEqn.A());
            volVectorField HbyA(constrainHbyA(rAU*UEqn.H(), U, p));
            surfaceScalarField phiHbyA
            (
                "phiHbyA",
                fvc::flux(HbyA)
              + fvc::interpolate(rAU)*fvc::ddtCorr(U, phi)
            );

            adjustPhi(phiHbyA, U, p);

            // Update the pressure BCs to ensure flux consistency
            constrainPressure(p, U, phiHbyA, rAU);

            // Non-orthogonal pressure corrector loop
            while (piso.correctNonOrthogonal())
            {
                // Pressure corrector

                fvScalarMatrix pEqn
                (
                    fvm::laplacian(rAU, p) == fvc::div(phiHbyA)
                );

                pEqn.setReference(pRefCell, pRefValue);

                pEqn.solve();

                if (piso.finalNonOrthogonalIter())
                {
                    phi = phiHbyA - pEqn.flux();
                }
            }

            #include "continuityErrs.H"

            U = HbyA - rAU*fvc::grad(p);
            U.correctBoundaryConditions();
        }

        runTime.write();

        Info<< "ExecutionTime = " << runTime.elapsedCpuTime() << " s"
            << "  ClockTime = " << runTime.elapsedClockTime() << " s"
            << nl << endl;
    }

    Info<< "End\n" << endl;

    return 0;
}
```

A typical OpenFOAM case

UK Atomic Energy Authority

- 0 directory
  - Contains initial and boundary conditions for variable fields
- Constant directory
  - Contains physical properties, turbulence models, gravity
- System
  - Numerical schemes
  - Solution parameters
  - Meshing
  - Partitioning



Simulation procedure

UK Atomic Energy Authority

1. Create or import mesh
   - Using BlockMesh or FluentMeshToFoam
2. Partition mesh for MPI
   - Use decomposePar
3. Run the solver
   - For solver module:
     - Mpirun –n 4 foamRun -parallel
   - For standalone
     - Mpirun -n 4 icoFoam –parallel
4. Reconstruct mesh (optional)
   - reconstructPar
5. Visualise mesh
   - Create .foam file
   - Touch viz.foam
   - View in paraview